Tarea M10 – RobertScience

Análisis SQL de Ventas

Base de Datos: AdventureWorks2022

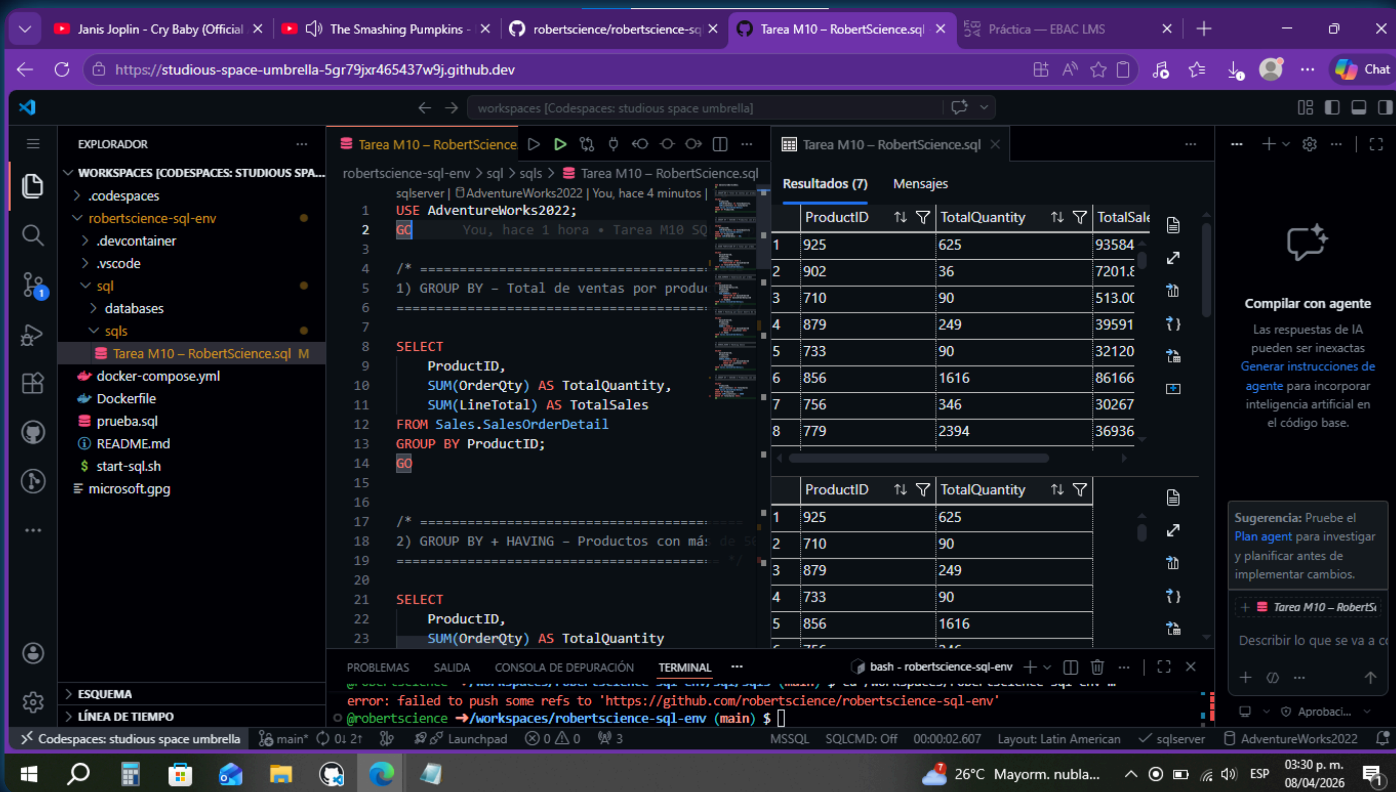
Objetivo

Aplicar consultas SQL para analizar información de ventas utilizando funciones de agregación y funciones de ventana para obtener métricas relevantes.

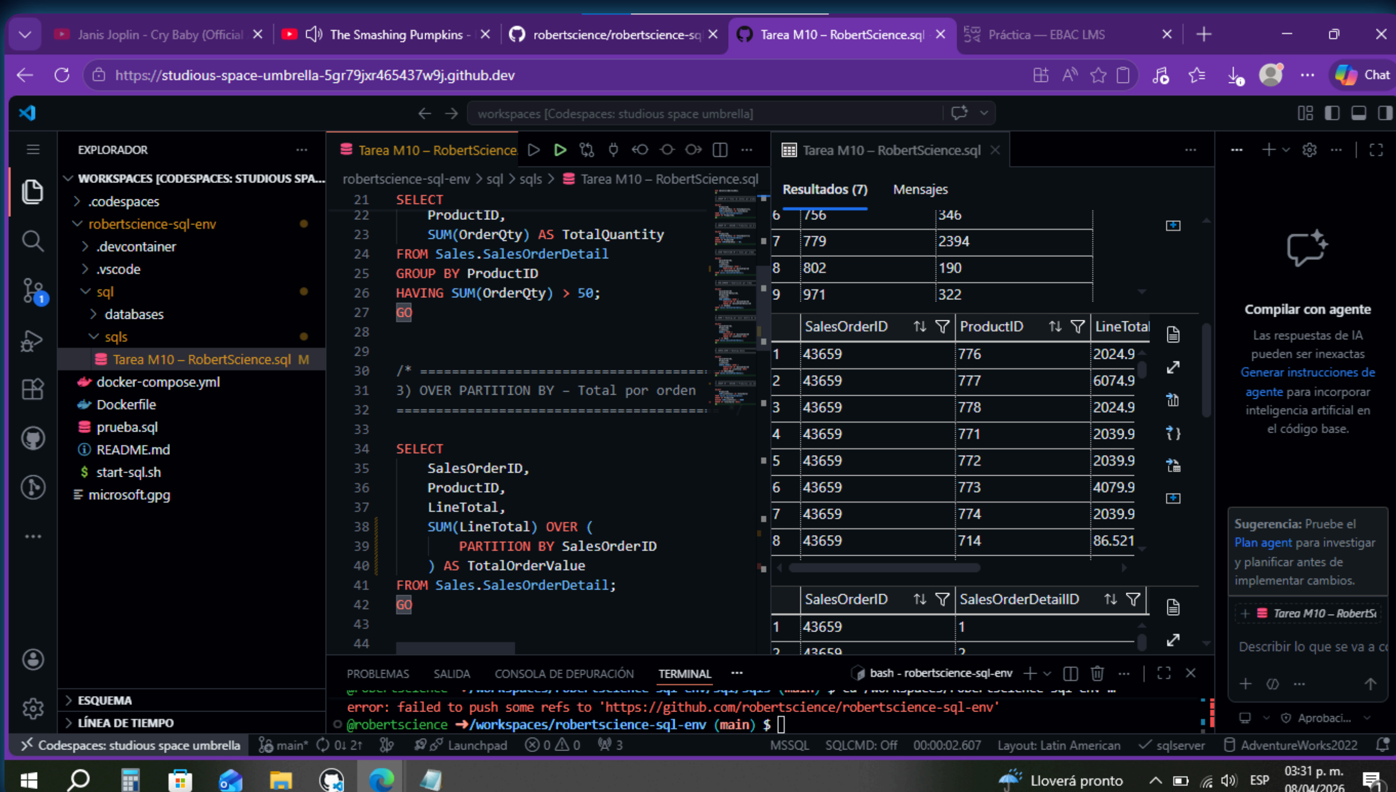
Descripción del Proyecto

Se analizaron datos de la tabla Sales.SalesOrderDetail para comprender el comportamiento de ventas, utilizando agrupaciones y funciones analíticas.

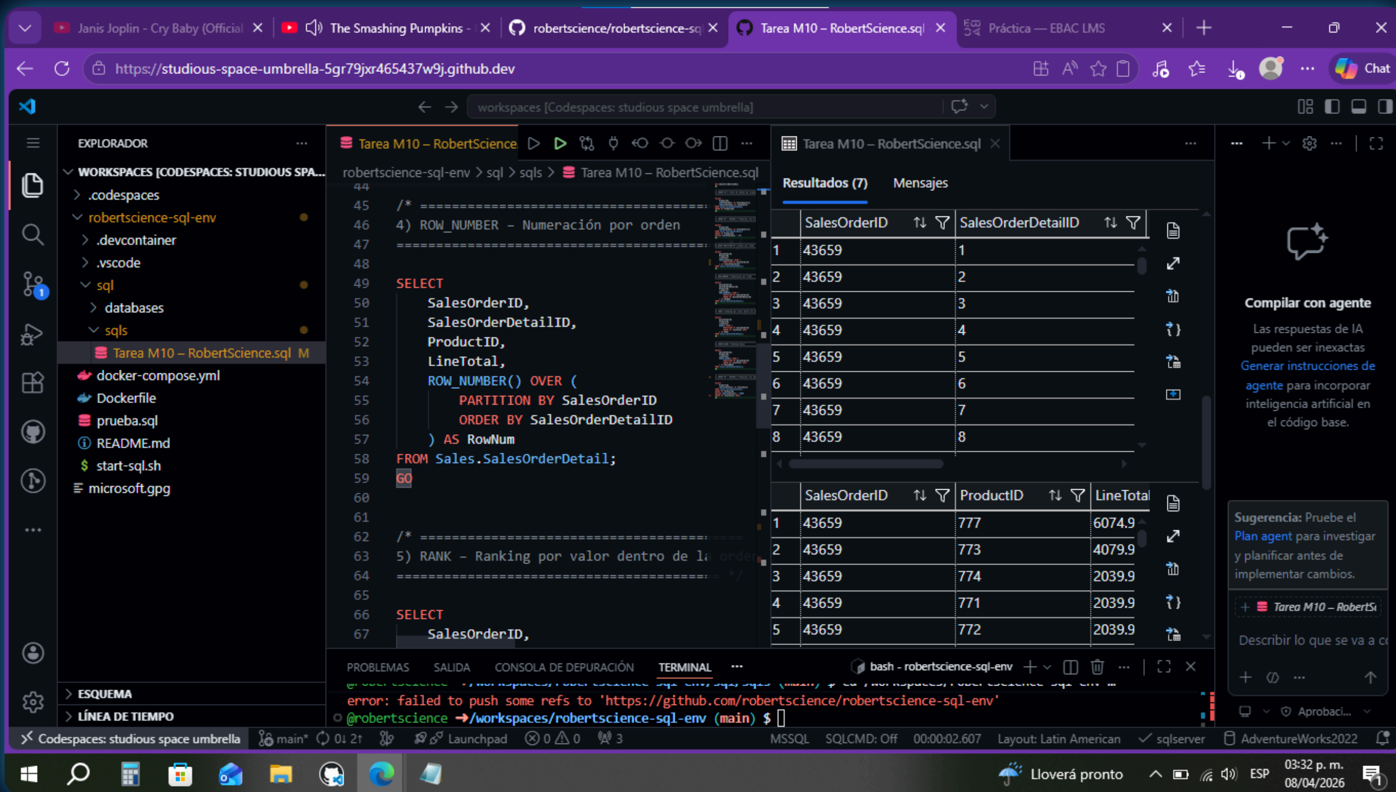
Resultados

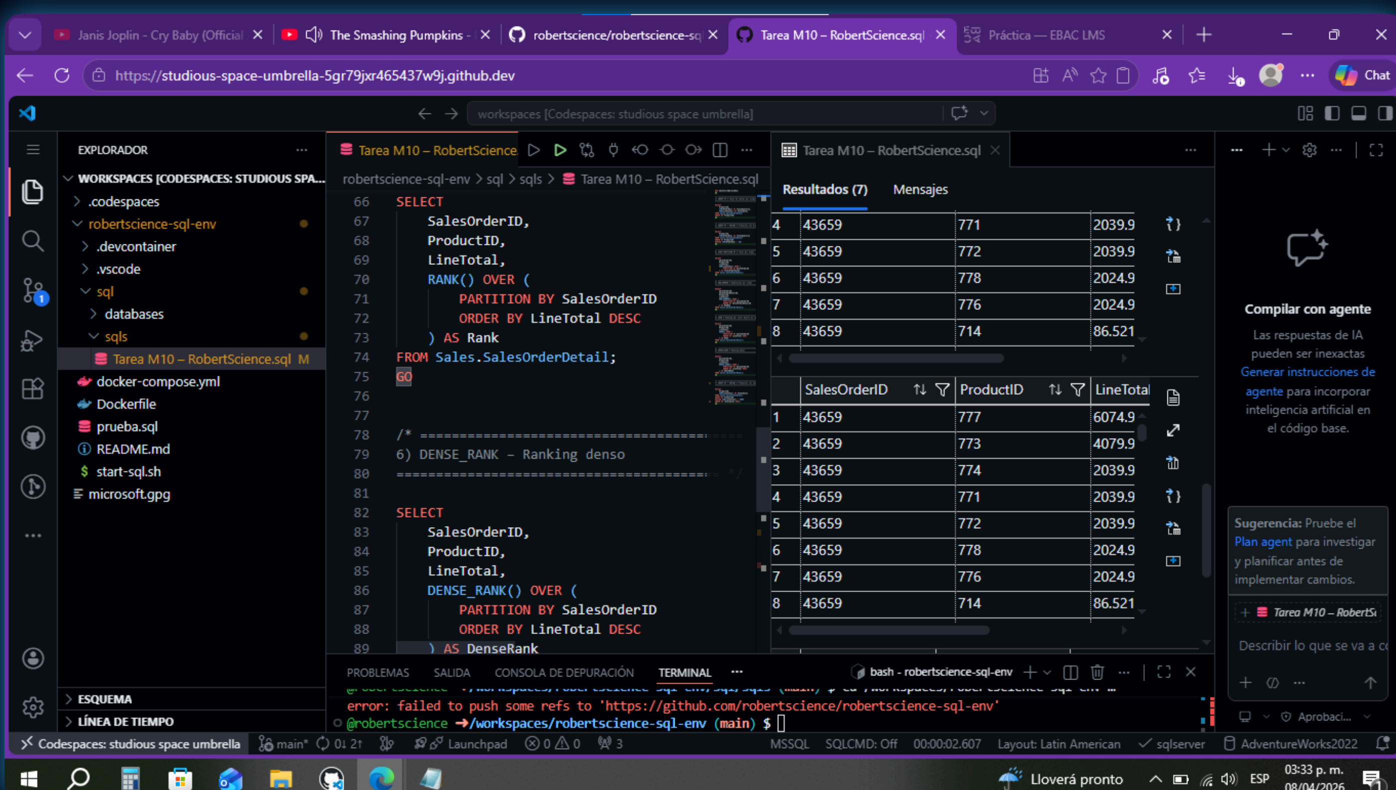


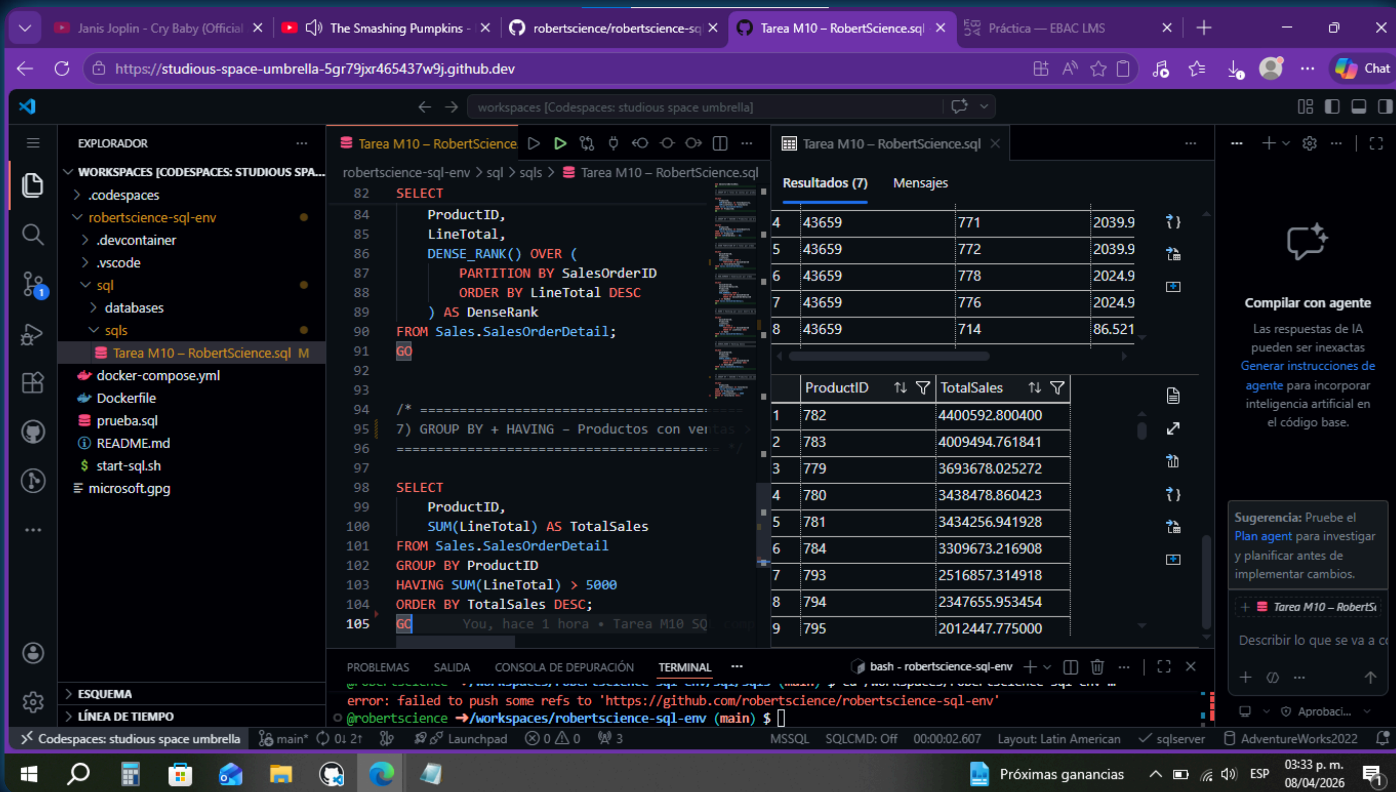
https://studious-space-umbrella-5gr79jxr465437w9j.github.dev
Tarea M10 – RobertScience.sql
USE AdventureWorks2022;
GO
/* ====
1) GROUP BY – Total de ventas por produ
SELECT
ProductID,
SUM(OrderQty) AS TotalQuantity,
SUM(LineTotal) AS TotalSales
FROM Sales.SalesOrderDetail
GROUP BY ProductID;
GO
2) GROUP BY + HAVING – Productos con más
SELECT
ProductID,
SUM(OrderQty) AS TotalQuantity
Resultados (7)
Mensajes
ProductID TotalQuantity TotalSale
1 925 625 93584
2 902 36 7201.8
3 710 90 513.00
4 879 249 39591
5 733 90 32120
6 856 1616 86166
7 756 346 30267
8 779 2394 36936
ProductID TotalQuantity
1 925 625
2 710 90
3 879 249
4 733 90
5 856 1616
Compilar con agente
error: failed to push some refs to 'https://github.com/robertscience/robertscience-sql-env'
@robertscience ➜/workspaces/robertscience-sql-env (main) $



https://studious-space-umbrella-5gr79jxr465437w9j.github.dev
Tarea M10 – RobertScience.sql
SELECT
ProductID,
SUM(OrderQty) AS TotalQuantity
FROM Sales.SalesOrderDetail
GROUP BY ProductID
HAVING SUM(OrderQty) > 50;
GO
/* ====
3) OVER PARTITION BY – Total por orden
SELECT
SalesOrderID,
ProductID,
LineTotal,
SUM(LineTotal) OVER (
PARTITION BY SalesOrderID
) AS TotalOrderValue
FROM Sales.SalesOrderDetail;
GO
Resultados (7)
Mensajes
6 756 346
7 779 2394
8 802 190
9 971 322
SalesOrderID ProductID LineTotal
1 43659 776 2024.9
2 43659 777 6074.9
3 43659 778 2024.9
4 43659 771 2039.9
5 43659 772 2039.9
6 43659 773 4079.9
7 43659 774 2039.9
8 43659 714 86.521
SalesOrderID SalesOrderDetailID
1 43659 1
Compilar con agente
error: failed to push some refs to 'https://github.com/robertscience/robertscience-sql-env'
@robertscience ➜/workspaces/robertscience-sql-env (main) $


Tarea M10 – RobertScience.sql
robertscience-sql-env > sql > sqls > Tarea M10 – RobertScience.sql
/* ==
4) ROW_NUMBER – Numeración por orden
==
SELECT
SalesOrderID,
SalesOrderDetailID,
ProductID,
LineTotal,
ROW_NUMBER() OVER (
PARTITION BY SalesOrderID
ORDER BY SalesOrderDetailID
) AS RowNum
FROM Sales.SalesOrderDetail;
GO
/* ==
5) RANK – Ranking por valor dentro de la orden
== */
SELECT
SalesOrderID,
Resultados (7)
Mensajes
SalesOrderID | SalesOrderDetailID
1 43659 1
2 43659 2
3 43659 3
4 43659 4
5 43659 5
6 43659 6
7 43659 7
8 43659 8
SalesOrderID | ProductID | LineTotal
1 43659 777 6074.9
2 43659 773 4079.9
3 43659 774 2039.9
4 43659 771 2039.9
5 43659 772 2039.9
error: failed to push some refs to 'https://github.com/robertscience/robertscience-sql-env'
@robertscience ➜/workspaces/robertscience-sql-env (main) $
Compilar con agente
03:32 p. m.
08/04/2026


SELECT
SalesOrderID,
ProductID,
LineTotal,
RANK() OVER (
PARTITION BY SalesOrderID
ORDER BY LineTotal DESC
) AS Rank
FROM Sales.SalesOrderDetail;
GO
/* ==
6) DENSE_RANK – Ranking denso
== */
SELECT
SalesOrderID,
ProductID,
LineTotal,
DENSE_RANK() OVER (
PARTITION BY SalesOrderID
ORDER BY LineTotal DESC
) AS DenseRank
Resultados (7)
Mensajes
4 43659 771 2039.9
5 43659 772 2039.9
6 43659 778 2024.9
7 43659 776 2024.9
8 43659 714 86.521
SalesOrderID | ProductID | LineTotal
1 43659 777 6074.9
2 43659 773 4079.9
3 43659 774 2039.9
4 43659 771 2039.9
5 43659 772 2039.9
6 43659 778 2024.9
7 43659 776 2024.9
8 43659 714 86.521
error: failed to push some refs to 'https://github.com/robertscience/robertscience-sql-env'
@robertscience ➜/workspaces/robertscience-sql-env (main) $
03:33 p. m.
08/04/2026



https://studious-space-umbrella-5gr79jxr465437w9j.github.dev
EXPLORADOR
WORKSPACES [CODESPACES: STUDIOUS SPA...
.codespaces
robertscience-sql-env
.devcontainer
.vscode
sql
databases
sqls
Tarea M10 – RobertScience.sql M
docker-compose.yml
Dockerfile
prueba.sql
README.md
start-sql.sh
microsoft.gpg
ESQUEMA
LÍNEA DE TIEMPO
robertscience-sql-env > sql > sqls > Tarea M10 – RobertScience.sql
SELECT
ProductID,
LineTotal,
DENSE_RANK() OVER (
PARTITION BY SalesOrderID
ORDER BY LineTotal DESC
) AS DenseRank
FROM Sales.SalesOrderDetail;
GO
7) GROUP BY + HAVING – Productos con ve
SELECT
ProductID,
SUM(LineTotal) AS TotalSales
FROM Sales.SalesOrderDetail
GROUP BY ProductID
HAVING SUM(LineTotal) > 5000
ORDER BY TotalSales DESC;
GO
Resultados (7)
Mensajes
4 43659 771 2039.9
5 43659 772 2039.9
6 43659 778 2024.9
7 43659 776 2024.9
8 43659 714 86.521
ProductID
TotalSales
1 782 4400592.800400
2 783 4009494.761841
3 779 3693678.025272
4 780 3438478.860423
5 781 3434256.941928
6 784 3309673.216908
7 793 2516857.314918
8 794 2347655.953454
9 795 2012447.775000
Compilar con agente
PROBLEMAS
SALIDA
CONSOLA DE DEPURACIÓN
TERMINAL
bash - robertscience-sql-env
error: failed to push some refs to 'https://github.com/robertscience/robertscience-sql-env'
@robertscience ➜/workspaces/robertscience-sql-env (main) $
Codespaces: studious space umbrella
MSSQL
SQLCMD: Off
Layout: Latin American
AdventureWorks2022
03:33 p. m.
08/04/2026

Script SQL

1) GROUP BY

```
SELECT ProductID, SUM(OrderQty) AS TotalQuantity, SUM(LineTotal) AS TotalSales FROM Sales.SalesOrderDetail
GROUP BY ProductID;
```

2) GROUP BY + HAVING

```
SELECT ProductID, SUM(OrderQty) AS TotalQuantity FROM Sales.SalesOrderDetail GROUP BY ProductID HAVING
SUM(OrderQty) > 50;
```

3) OVER PARTITION BY

```
SELECT SalesOrderID, ProductID, LineTotal, SUM(LineTotal) OVER (PARTITION BY SalesOrderID) AS
TotalOrderValue FROM Sales.SalesOrderDetail;
```

4) ROW_NUMBER

```
SELECT SalesOrderID, SalesOrderDetailID, ProductID, LineTotal, ROW_NUMBER() OVER ( PARTITION BY
SalesOrderID ORDER BY SalesOrderDetailID ) AS RowNum FROM Sales.SalesOrderDetail;
```

5) RANK

```
SELECT SalesOrderID, ProductID, LineTotal, RANK() OVER ( PARTITION BY SalesOrderID ORDER BY LineTotal DESC
) AS Rank FROM Sales.SalesOrderDetail;
```

6) DENSE_RANK

```
SELECT SalesOrderID, ProductID, LineTotal, DENSE_RANK() OVER ( PARTITION BY SalesOrderID ORDER BY LineTotal
DESC ) AS DenseRank FROM Sales.SalesOrderDetail;
```

7) HAVING > 5000

```
SELECT ProductID, SUM(LineTotal) AS TotalSales FROM Sales.SalesOrderDetail GROUP BY ProductID HAVING
SUM(LineTotal) > 5000 ORDER BY TotalSales DESC;
```

Conclusión

En esta práctica comprendí cómo analizar datos utilizando diferentes herramientas de SQL. Las funciones de agregación permiten resumir información, mientras que las funciones de ventana ofrecen una visión más detallada del comportamiento de los datos dentro de cada grupo.

Este tipo de consultas es fundamental para el análisis de datos en entornos reales, ya que permite obtener información clara y estructurada para la toma de decisiones.